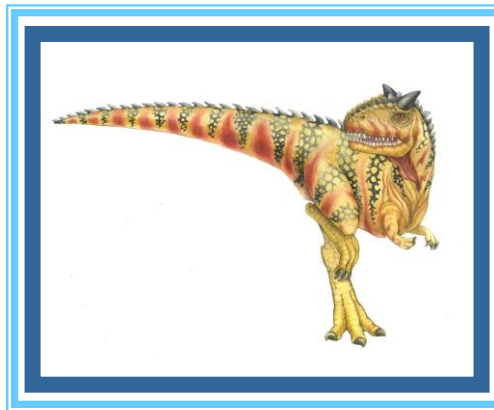


Unidad 4: Threads





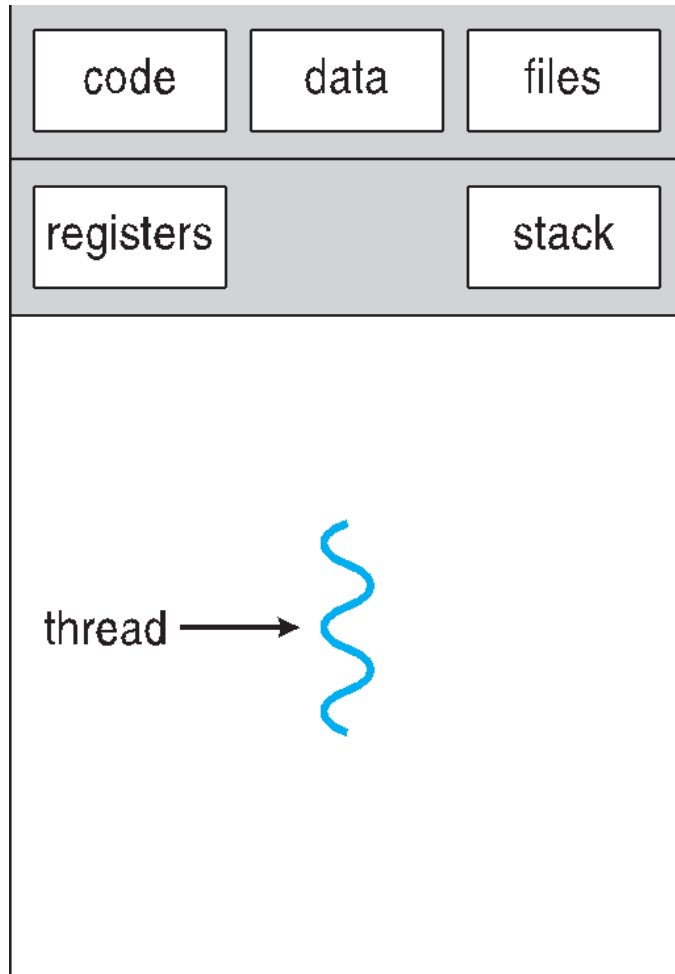
Unidad 4: Threads

- Programación Multicore
- Modelos de Multithreading
- Librerías de Thread
- Ejemplos en Linux

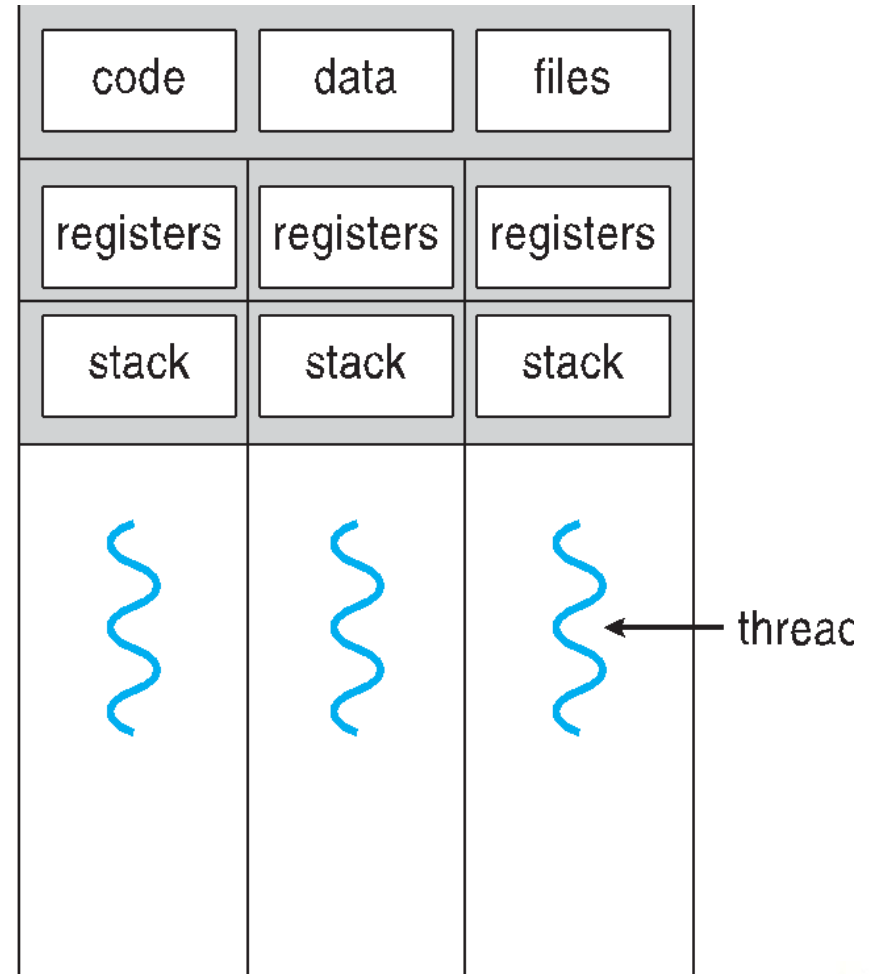




Procesos de uno y muchos threads



single-threaded process



multithreaded process





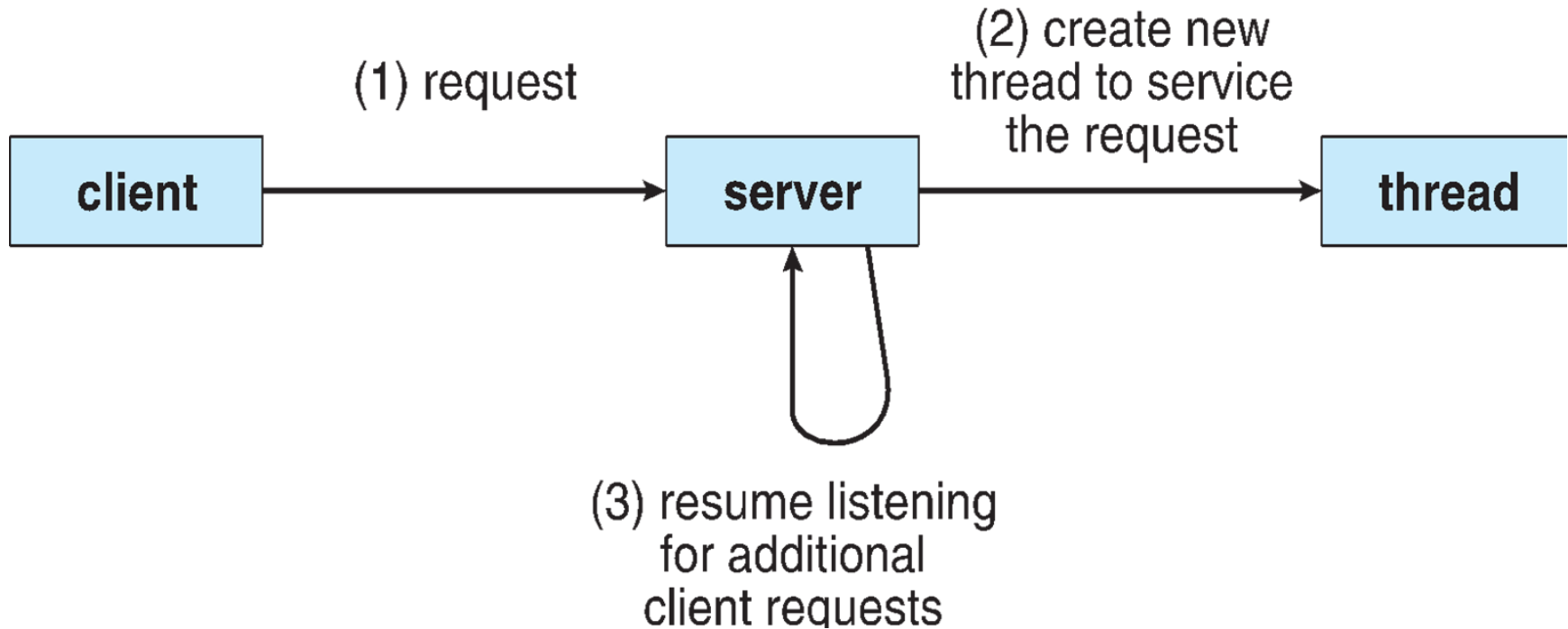
Motivación

- La mayoría de las aplicaciones modernas son multiproceso.
- Múltiples tareas dentro de la aplicación pueden ser implementadas en threads separados
 - Actualizar la pantalla
 - Obtener información
 - Corrección ortográfica
 - Responder una solicitud de red
- La creación del proceso es pesada, mientras que la creación de threads es liviana
- Puede simplificar el código, aumentar la eficiencia.
- Los Kernels generalmente son multithreaded





Arquitectura servidor multithread





Beneficios

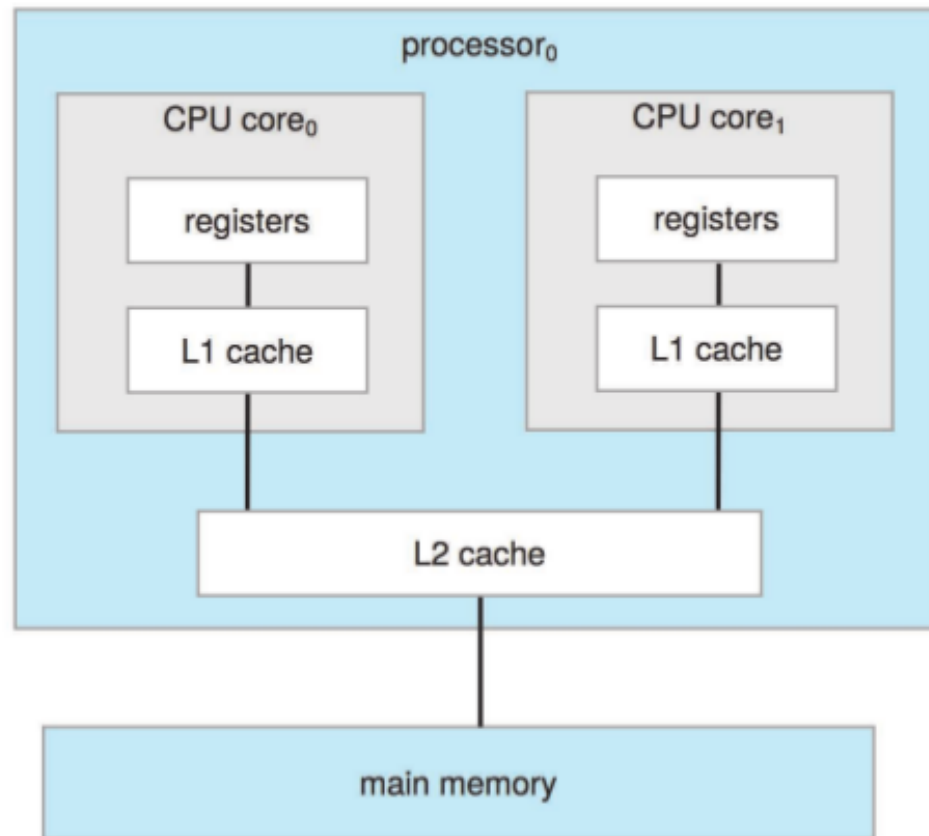
- **Capacidad de respuesta:** puede permitir la ejecución continua si se bloquea parte del proceso, especialmente importante para las interfaces de usuario
- **El intercambio de recursos** – los hilos comparten recursos de proceso, más fáciles que la memoria compartida o el paso de mensajes.
- **Economía** – es más barato que la creación de procesos, el cambio de contextos genera menos overhead
- **Escalabilidad** – el proceso puede aprovechar las arquitecturas multiprocesador.





Programación Multicore

- Sistemas **Multicore** (multinucleo) o **multiprocessor** (multiprocesador)





Programación Multicore

- Sistemas **Multicore** (multinucleo) o **multiprocessor** (multiprocesador)
- Estos sistemas implican al desarrollador:
 - Dividir actividades
 - Balance
 - División de los datos
 - Dependencia de los datos
 - Testing y debugging
- **Parallelism** (paralelismo) implica que un sistema puede realizar más de una tarea simultáneamente
- **Concurrency** (conurrencia) Soporta más de una tarea progresando
 - Single processor / core, el planificador que proporciona conurrencia





Programación Multicore (Cont.)

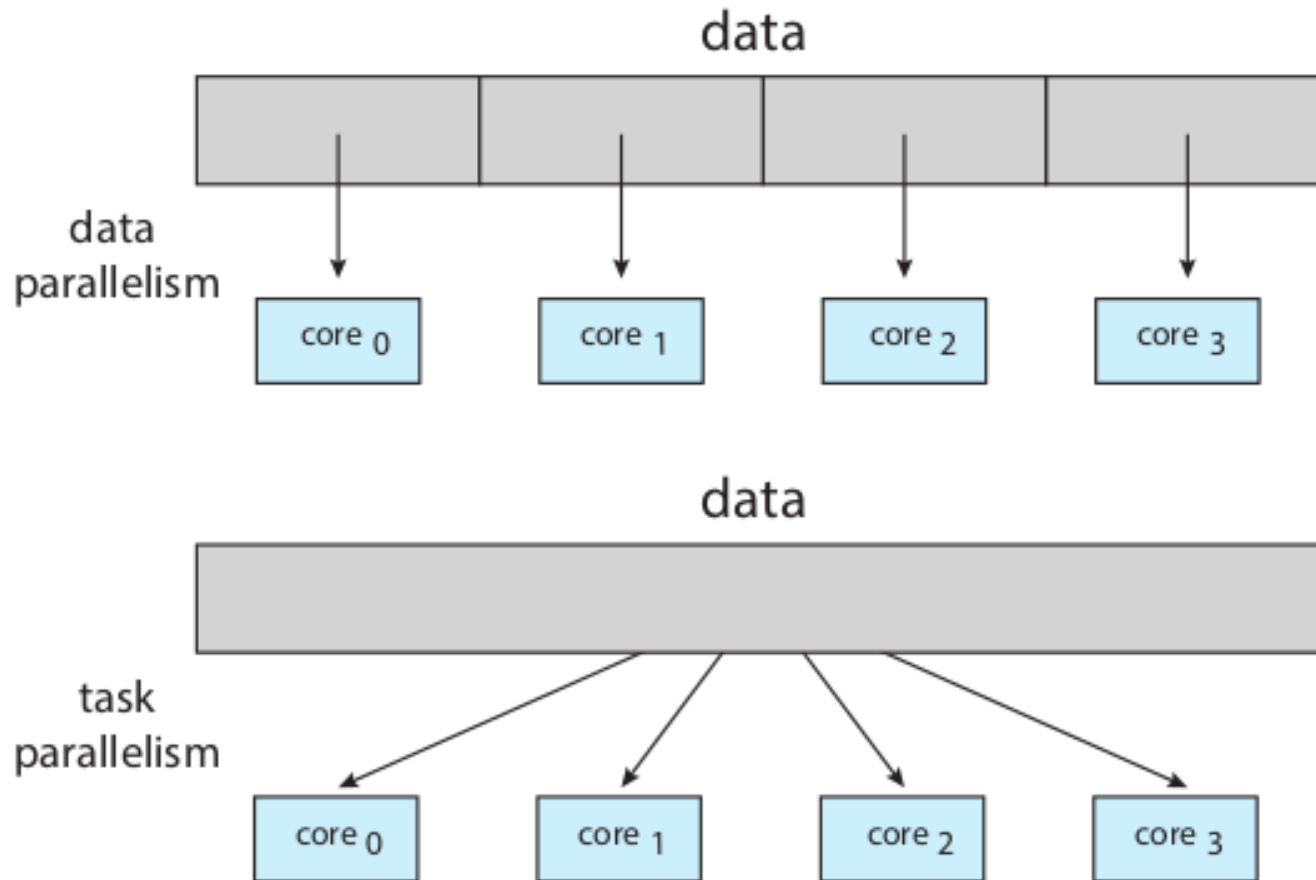
- Tipos de paralelismo
 - **Paralelismo de datos** – distribuye subconjuntos de los mismos datos a través de múltiples núcleos, la misma operación en cada uno
 - **Paralelismo de tareas** – distribuye los hilos a través de núcleos, cada hilo realiza una operación única.





Programación Multicore (Cont.)

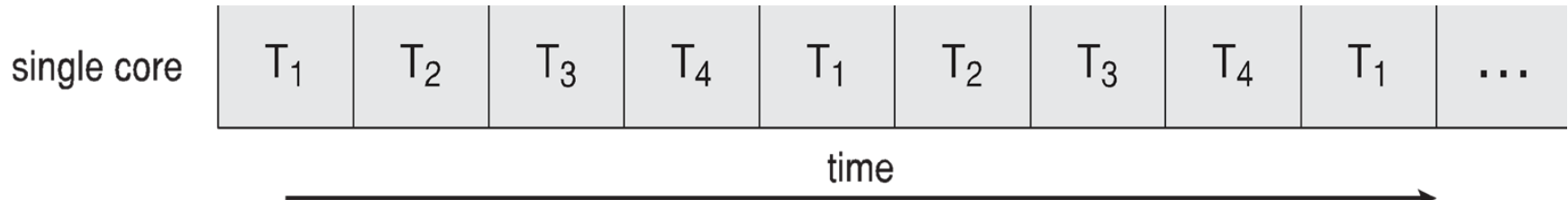
- Tipos de paralelismo



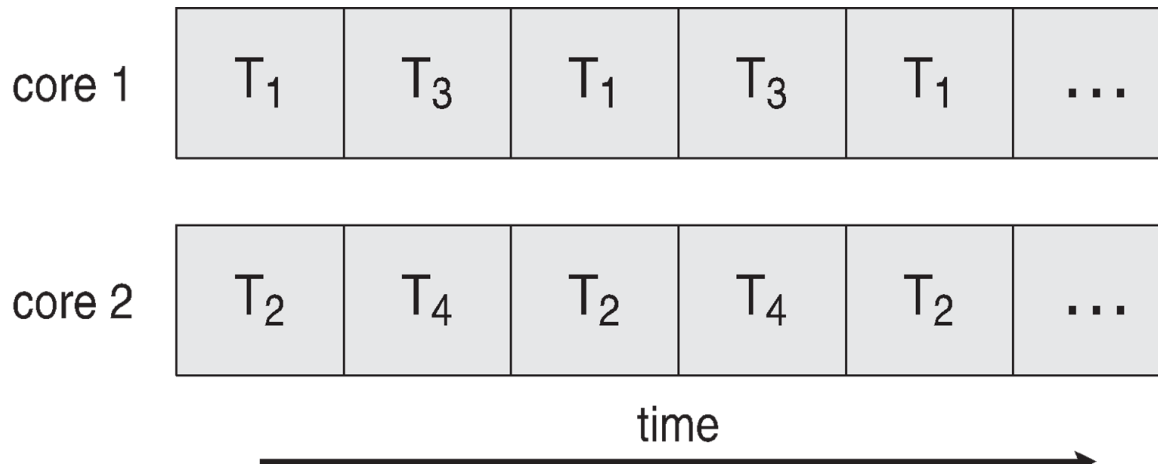


Concurrencia vs. Paralelismo

- Ejecución concurrente en sistema de un solo núcleo:



- Paralelismo en un sistema multi-core:





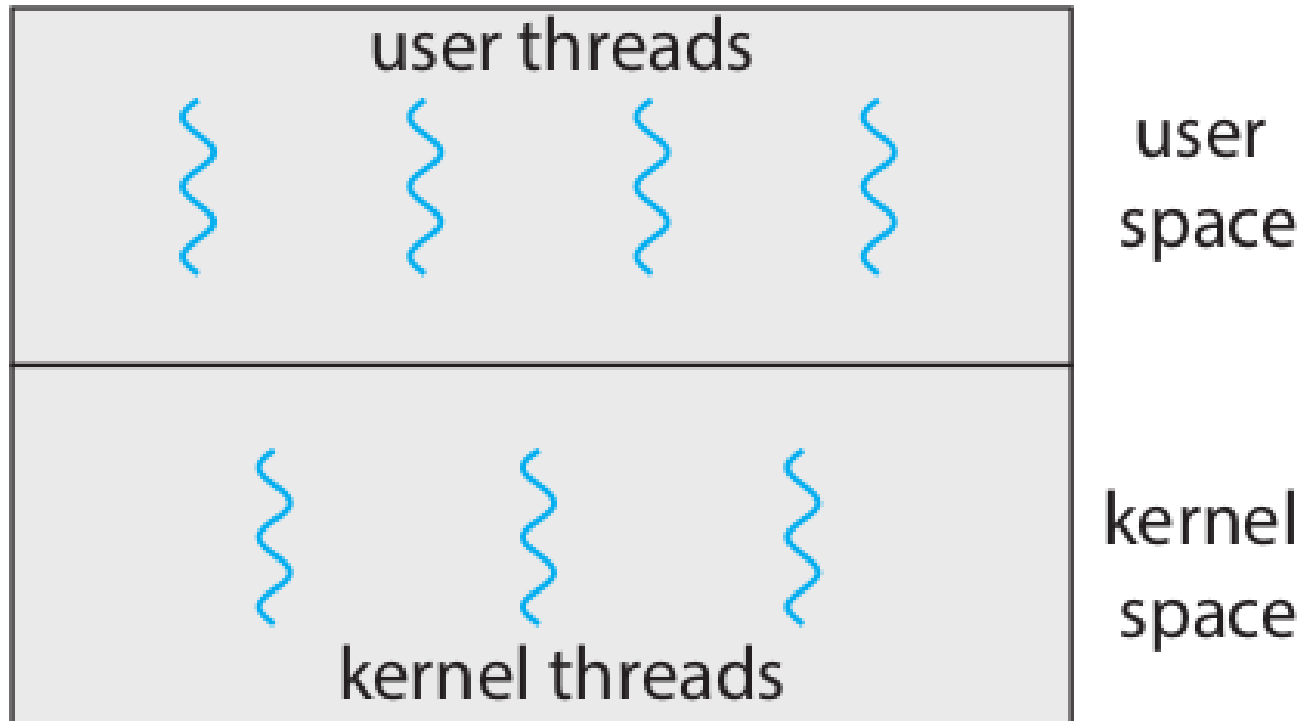
Threads de usuario y de Kernel

- **Threads de Usuario** - el manejo se realiza a nivel de usuario a través de librerías.
- Tres librerías para el manejo de hilos:
 - POSIX **Pthreads**
 - Windows threads
 - Java threads
- **Kernel threads** - Soporte a nivel Kernel
- Prácticamente todos los SO de propósito general lo permiten:
 - Windows
 - Solaris
 - Linux
 - Mac OS X





Threads de usuario y de Kernel





Threads de usuario y de Kernel

Kernel threads son hijos del proceso PID 2 (kthreadd), y se los puede visualizar con el siguiente comando

```
@debian:~$ ps --ppid 2 -p 2 -o uname,pid,ppid,cmd,cls
```

USER	PID	PPID	CMD	CLS
root	2	0	[kthreadd]	TS
root	3	2	[ksoftirqd/0]	TS
root	5	2	[kworker/0:0H]	TS
root	7	2	[rcu_sched]	TS

...





Threads de usuario y de Kernel

Para ver los threads creados por un proceso dado el PID ejecutar:

```
ps -T -p <pid>
```





Modelos Multithread

- Muchos a uno
- Uno a uno
- Muchos a muchos





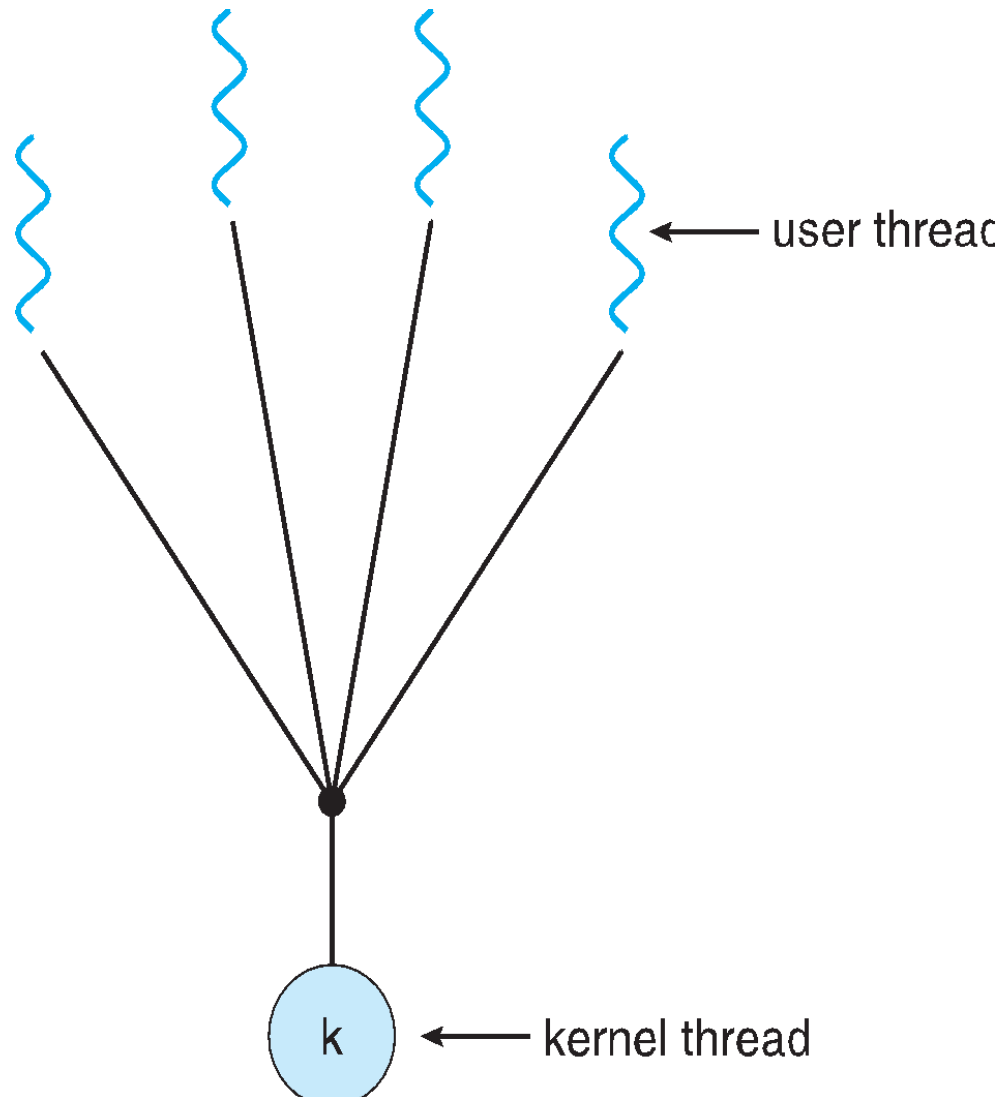
Muchos a uno

- Muchos user-threads son mapeados a un único kernel-thread
- Múltiples hilos no pueden correr en paralelo en sistema muticore porque solo uno puede estar en el kernel a la vez
- Pocos sistemas actualmente utilizan este modelo.
- Ejemplos:
 - Solaris Green Threads
 - GNU Portable Threads





Muchos a uno





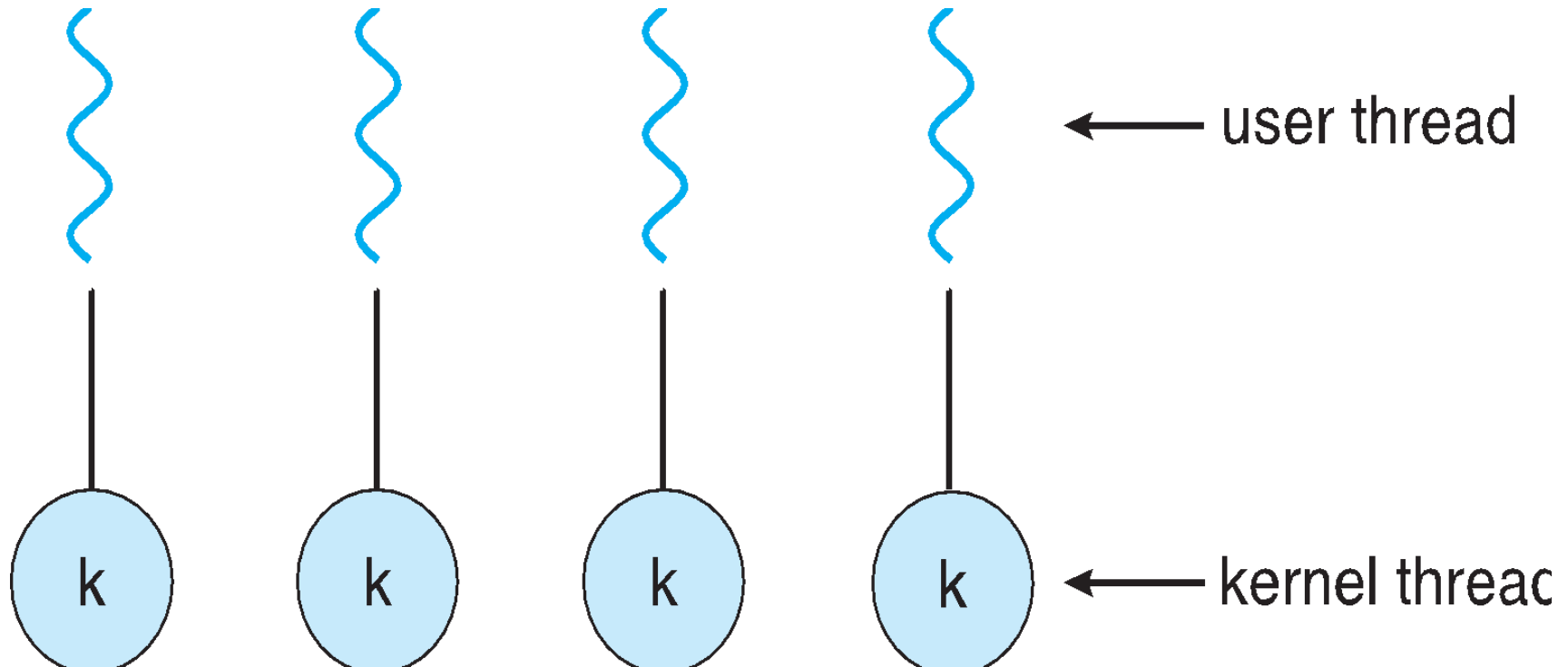
Uno a uno

- Cada thread de usuario es asignado a un kernel thread
- La creación de thread de usuario implica la creación de un kernel thread
- Mas concurrencia que muchos a uno
- El número de threads por proceso a veces restringido debido al overhead
- Ejemplos
 - Windows
 - Linux
 - Solaris 9 and later





Uno a uno





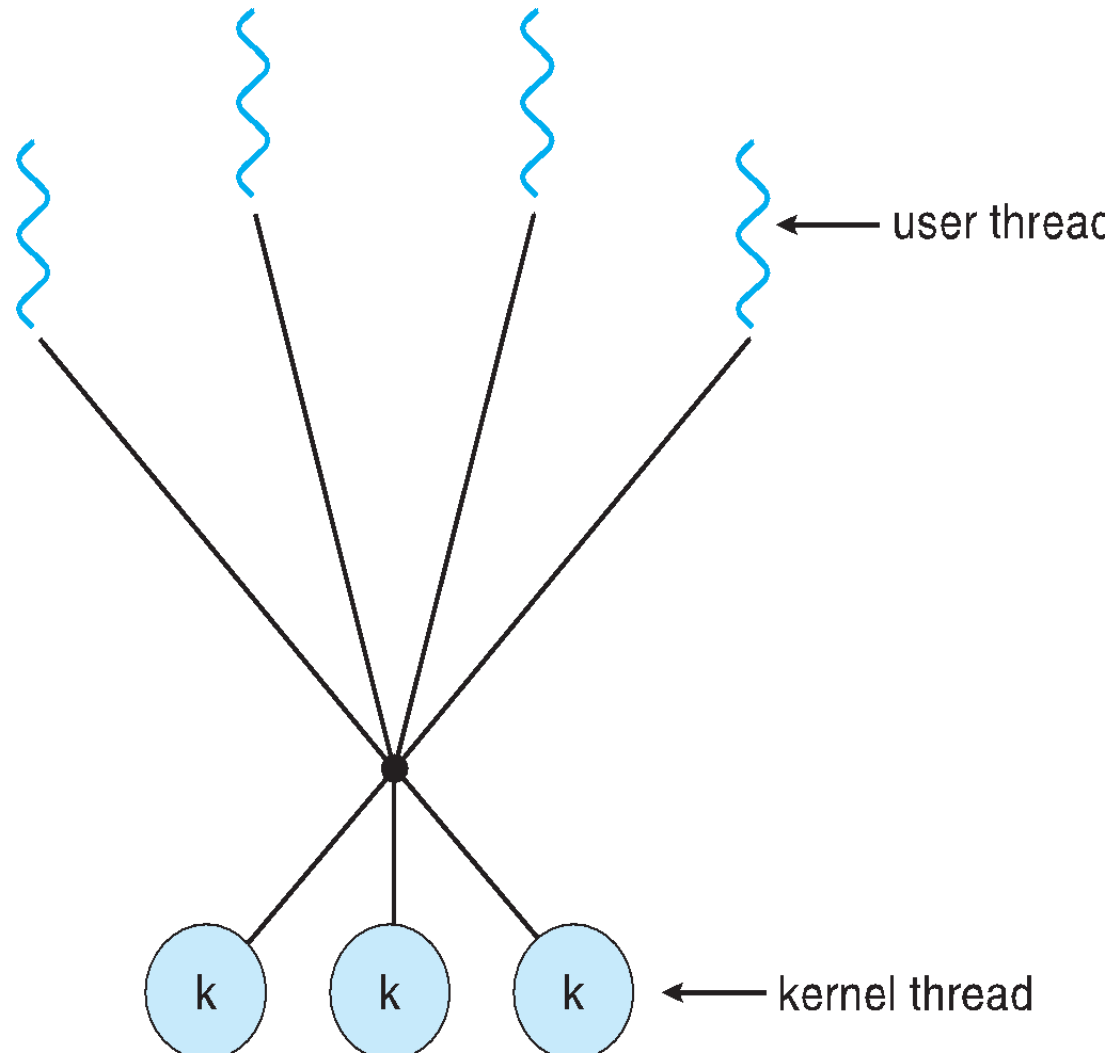
Muchos a muchos

- Permite que muchos threads de usuario se asocien a muchos de kernel threads
- Permite que el sistema operativo cree una cantidad suficiente de kernel threads
- Solaris a partir de la version 9
- Windows con *ThreadFiber* package





Muchos a muchos





Librerías de Threads

- Proveen al desarrollador con una API (application programming interface) para la creación y manejo de threads
- Existen dos formas principales de implementación
 - Librerías completamente en el espacio (nivel) de usuario.
 - Librerías a nivel Kernel soportadas por el SO





Pthreads

- Puede proveerse a threads de nivel-usuario o nivel-kernel
- A POSIX standard (IEEE 1003.1c) API para la creación y sincronización de threads
- API es para ***Especificación***, no ***implementación***
- API especifica el comportamiento de la librería para threads, la implementación depende del desarrollo de la biblioteca.
- Es comun en SO UNIX (Solaris, Linux, Mac OS X)





Ejemplo Pthreads

[link al codigo](#)

```
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>
#include <stdint.h>

int sum; /* esta variable es compartida por los thread(s) */

void* funcion(void* param); /* los threads llaman a esta funcion */

int main(int argc, char *argv[])
{
    pthread_t tid; /* el identificador del thread */
    pthread_attr_t attr; /* conjunto de atributos del thread */

    pthread_attr_init(&attr); /* setear los atributos por defecto*/

    int parametro_funcion=4;

    /* crear el thread */
    pthread_create(&tid, &attr, funcion, (void *) (intptr_t) parametro_funcion);
    pthread_join(tid,NULL); /* esperar a la salida del thread */

    printf("suma es = %d \n",sum);
}
```





Ejemplo Pthreads (Cont.)

```
/* el thread ejecuta esta funcion */  
void *funcion(void *param)  
{  
    int i;  
    int upper = (intptr_t) param;  
  
    sum = 0;  
    for (i = 1; i <= upper; i++){  
        sum += i;  
    }  
    pthread_exit(0);  
}
```





Crear ó esperar 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```





Linux Threads

- En Linux se refieren a los threads como **tasks** (tareas)
- La creación threads es con la librería pthread, internamente se utiliza la system call **clone()**
- **clone()** permite que una tarea hija comparta información con su tarea padre (process), por ejemplo puede compartir la siguiente información (determinada por flags):

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.



Fin de Unidad 4

